

Dedco. Marketplace béninoise

Document de référence sur l'architecture technique et produit de Dedco, la marketplace béninoise de l'aménagement intérieur. Stack Next.js 16, séparation stricte des rôles, moteur métier brief-artisan, 60+ pages dynamiques. Synthèse des 21+ corrections d'audit et des chantiers ouverts.

Préparé par Équipe Dedco · Version 1.0 · Date 3 juillet 2026

Table des matières

1. Vision & contexte	4
2. Stack technique & infrastructure	5
Déploiement & CI/CD	5
3. Architecture SPA & routing interne	7
Type AppRoute — discriminated union	7
Code splitting	7
Bridge AppRoute ↔ Route legacy	7
4. Design system Dedco	8
Typographie	8
Classes utilitaires custom	8
5. Rôles utilisateurs & permissions	9
ROLE_DASHBOARD — mapping rôle → page	9
Séparation prestataires / clients	9
6. Route guarding & redirection silencieuse	10
Trois guards complémentaires	10
ROLE_HOME_PAGE — redirection par défaut	10
Pages partagées — accessibles multi-rôles	10
7. Stores Zustand — état global	11
Persistance sélective	11
Navigation programmatique	11

8. Shell & layouts dashboard	12
DashboardSidebar — mémorisée et stable	12
Layout h-screen + overflow-hidden	12
allowExitToSite — verrou prestataire	12
sharedPages — pages wrappées selon rôle	12
9. Moteur métier brief-artisan	13
11 statuts de brief artisan	13
10. Workflows métier	15
Brief artisan — cycle complet	15
Brief designer — prestation de service	15
Paiement escrow & garantie 1.5%	15
Page Mes Projets — vue unifiée client	15
11. Pages partagées — affichage role-aware	16
Helper getBackToProjets — boutons retour role-aware	16
Carte partenaire — inversée selon rôle	17
12. Messagerie & notifications	18
Bannière de sécurité — « Échange protégé par Dedco »	18
NotificationStore — notifications par rôle	18
Marquer comme lu + Tout lire	18
13. Arborescence des fichiers	19
14. Audit & corrections récentes	20
15. Roadmap & chantiers ouverts	21

P1-4 — Migration vraies routes App Router	21
P1-6 — Consolidation modules data	21
P3 — next/image sur 36 raw 	21
Backend réel — Prisma + API routes	21
Tests & CI	21

16. Conclusion 22

1. Vision & contexte

Dedco est la marketplace béninoise de l'aménagement intérieur qui connecte directement les clients souhaitant meubler ou redécorer leur intérieur avec les designers et artisans locaux. La plateforme opère depuis Cotonou, Porto-Novo et Parakou, et met en valeur un savoir-faire typiquement béninois : bois iroko, rotin tressé, bogolan, wax. L'enjeu est double — d'une part offrir aux clients un accès direct à des créateurs vérifiés avec paiement sécurisé et livraison tracée, d'autre part fournir aux artisans et designers un canal de vente professionnel et un outil de gestion de leur activité.

Le produit digital se décompose en quatre espaces distincts : un **site public** (marketplace, inspirations, fiches artisans/designers, magazine) accessible à tous les visiteurs ; un **espace client** pour le suivi des commandes, briefs et projets ; et trois **dashboards prestataires** (artisan, designer, maison déco) qui fonctionnent comme des back-offices spécialisés par métier. Un tableau de bord administrateur complète l'ensemble pour la modération et la gestion plateforme.

Cette architecture multi-acteurs constitue le cœur du produit : chaque rôle dispose d'un parcours adapté à sa logique métier, et les transitions entre espaces sont strictement contrôlées pour éviter toute confusion. Les prestataires sont ainsi enfermés dans leur dashboard dédié, sans accès au site public, tandis que les clients naviguent librement sur la marketplace et accèdent à leurs pages de suivi.

5	60+	154	43
Rôles utilisateurs	Pages dynamiques	Fichiers TS/TSX	Commits (30 jours)

2. Stack technique & infrastructure

Le projet repose sur une stack Next.js moderne, choisie pour son écosystème mature, son support TypeScript natif et son déploiement serverless sur Vercel. Le build utilise Turbopack pour des temps de compilation réduits, et le code splitting est poussé via 84 imports dynamic() pour les pages secondaires. Les pages critiques (Home, Marketplace, Product, Scene) sont chargées en eager pour optimiser le LCP.

Technologie	Version	Rôle
Next.js	16.1.3 (Turbopack)	Framework React, App Router, SSR
TypeScript	5.x	Typage statique strict
Tailwind CSS	4.x	Styling utility-first + tokens custom
shadcn/ui + Radix UI	Dernière	Composants accessibles (Dialog, Drawer, etc.)
Zustand	5.x	State management avec middleware persist
Prisma ORM	6.11	Schema SQLite (User, Post — backend minimal)
Framer Motion	11.x	Animations de page, transitions, micro-interactions
Lucide Icons	Dernière	Iconographie cohérente
Vercel	Auto-deploy	Hébergement serverless, preview par branche

Déploiement & CI/CD

Le code source est hébergé sur GitHub à l'adresse github.com/Golden-003/Dedco. Chaque push sur la branche main déclenche un build Vercel automatique avec déploiement en production sur **dedco-h8zw.vercel.app**. Les branches secondaires bénéficient de preview URLs isolées. La pipeline inclut un lint ESLint et un type-check TypeScript, ainsi qu'un build de validation avant publication. Aucune base de données n'est actuellement active en production — l'application fonctionne en mode démo avec données mockées et persistance localStorage.

État actuel : 1 route API (/api hello-world). Le backend réel (auth, products, orders) a été retiré — l'app fonctionne en SPA pur avec données mockées. La migration vers un backend Prisma + API routes est planifiée en P1-4.

3. Architecture SPA & routing interne

Dedco utilise une architecture SPA mono-route : une seule route App Router (/ dans src/app/page.tsx) sert l'ensemble de l'application. La navigation interne est gérée par un store Zustand qui maintient l'objet route courant (page + paramètres), et un switch dans DedcoRouter résout la page à afficher. Cette approche a été choisie pour sa simplicité de mise en œuvre et la fluidité des transitions (pas de rechargement, pas de layout shift), au prix d'une migration future vers de vraies routes App Router planifiée en P1-4.

Type AppRoute — discriminated union

Toutes les routes possibles sont décrites par une union TypeScript dans src/lib/store.ts. Le typage garantit que chaque navigation passe les bons paramètres (id, briefId, projectId, proposalId, etc.).

```
type AppRoute =  
  | { page: 'home' }  
  | { page: 'product'; id: number }  
  | { page: 'projet-artisan-detail'; projectId: string }  
  | { page: 'brief-designer'; designerId: number }  
  | { page: 'messages'; conversationId?: string }  
  | ... // 60+ variantes
```

Code splitting

Les pages sont chargées selon deux stratégies : **eager** pour les pages critiques LCP (Home, Marketplace, Product, Scene, Inspirations, Designers) et **lazy** via dynamic() pour les 80+ pages secondaires. Cette répartition réduit le bundle initial de ~40% tout en gardant les pages clés instantanément disponibles. Les pages dashboard sont chargées en eager pour éviter tout flash lors de la navigation intra-dashboard.

Bridge AppRoute ↔ Route legacy

Pour préserver la compatibilité avec les composants hérités qui utilisent l'ancien type Route (nom-based), deux fonctions bridge (appRouteToRoute et routeToAppRoute) assurent la conversion. Le Navbar et le BottomNav consomment encore ce type legacy.

4. Design system Dedco

L'identité visuelle de Dedco repose sur trois couleurs brand ancrées dans l'esthétique béninoise : l'**amber** (#BF793B) pour l'accent principal et les CTA, la **terracotta** (#A6442E) pour les états d'alerte et de validation urgente, et le **forest** (#548C45) pour les confirmations et les indicateurs positifs. Ces trois teintes sont déclinées en variantes pale pour les fonds et en variantes dark pour le texte accentué. Le fond crème (#FAF7F2) apporte chaleur et lisibilité, tandis que l'encre (#1E1813) sert de texte primaire.

Token	Hex	Usage
--amber	#BF793B	CTA, accents, liens, badges
--amber-pale	#FEF5E9	Fonds hover, sélection
--terracotta	#A6442E	Alertes, litiges, déconnexion
--forest	#548C45	Succès, paiements, en ligne
--bg-cream	#FAF7F2	Fond principal
--bg-card	#FFFFFF	Cartes, surfaces
--text-1	#1E1813	Texte primaire (encre)
--text-3	#7A7068	Texte muted (WCAG AA)

Typographie

Deux familles coexistent : **Quache** pour les titres et mots isolés (display), et **Plus Jakarta Sans** pour le corps de texte et les chiffres. Une astuce unicode-range exclut les chiffres (U+0030-0039) de Quache pour forcer le fallback vers Jakarta, dont les chiffres sont plus lisibles. Les 5 weights Quache sont embarqués en base64 dans globals.css pour éviter toute requête supplémentaire.

Classes utilitaires custom

Une bibliothèque de classes `.dedco-*` centralise les patterns récurrents : `.dedco-btn`, `.dedco-btn-primary`, `.dedco-card`, `.dedco-badge-*`, `.dedco-fade-in`. Ces classes garantissent la cohérence visuelle et réduisent la duplication CSS. Un brand book PDF de 55 pages documente l'ensemble du système et a été livré séparément.

5. Rôles utilisateurs & permissions

Cinq rôles coexistent dans Dedco, chacun avec un périmètre d'action strictement délimité. La règle d'or est la **séparation stricte** entre clients (qui consomment le site public) et prestataires (qui sont enfermés dans leur dashboard dédié). Cette séparation évite la confusion entre interfaces et garantit que chaque acteur voit l'application à travers le prisme de son métier.

Rôle	Page d'accueil	Accès site public	Dashboard dédié
Client	home (marketplace)	Oui	Non — navbar publique
Artisan	artisan-dashboard	Non (verrouillé)	Oui — sidebar artisan
Designer	designer-dashboard	Non (verrouillé)	Oui — sidebar designer
Maison déco	maison-dashboard	Non (verrouillé)	Oui — sidebar maison
Admin	admin-dashboard	Oui	Oui — sidebar admin

ROLE_DASHBOARD — mapping rôle → page

Une table `ROLE_DASHBOARD` dans `layout.tsx` mappe chaque rôle vers sa page d'accueil dédiée. Le menu utilisateur (`UserMenu`) et le drawer mobile utilisent cette table pour router l'utilisateur connecté vers son espace. Le libellé et l'icône associés (`ROLE_LABEL`, `ROLE_ICON`) sont affichés dans le dropdown de profil et la sidebar du dashboard.

Séparation prestataires / clients

La séparation est appliquée à deux niveaux. Au niveau du **shell** (`page.tsx`), la navbar publique, le bottom-nav et le footer sont masqués pour tout rôle prestataire — peu importe la page consultée. Au niveau du **route guarding** (`dedco-router.tsx`), toute tentative d'accès à une page publique par un prestataire déclenche une redirection silencieuse vers son dashboard. Cette double barrière garantit qu'aucun prestataire ne puisse naviguer sur la marketplace, inspirations ou autre page client.

Règle métier : les prestataires (artisan, designer, maison déco) n'ont accès qu'à leur dashboard. Le client navigue sur le site public et accède à ses pages de suivi. L'admin est le seul rôle hybride — il garde l'accès au site public ET à son tableau de bord d'administration.

6. Route guarding & redirection silencieuse

Le route guarding de Dedco a été repensé pour **supprimer l'écran « Accès restreint »** (GuardBlocked) qui s'affichait auparavant quand un utilisateur tentait d'accéder à une page non autorisée. À la place, une redirection silencieuse est effectuée via `useEffect + navigate()`. Le temps de la redirection (1 frame), le composant rend null pour éviter tout flash de contenu non autorisé.

Trois guards complémentaires

Guard	Condition	Action
requiredRole	Page dans ARTISAN_PAGES / DESIGNER_PAGES / ADMIN_PAGES / maison-dashboard	Redirige si rôle ≠ requis
needsAuth	Page dans AUTH_REQUIRED_PAGES (18 pages)	Redirige vers login si non connecté
isLockedOut	Rôle prestataire + page publique	Redirige vers dashboard du rôle

ROLE_HOME_PAGE — redirection par défaut

```
const ROLE_HOME_PAGE: Record<UserRole, AppRoute['page']> = {
  client: 'home',
  artisan: 'artisan-dashboard',
  designer: 'designer-dashboard',
  admin: 'admin-dashboard',
  maison: 'maison-dashboard',
};
```

Pages partagées — accessibles multi-rôles

Certaines pages de détail sont **partagées** entre client et prestataire : projet-artisan-detail, projet-designer-detail, brief-artisan-detail, brief-designer-detail. Elles ont été retirées des sets ARTISAN_PAGES / DESIGNER_PAGES et ajoutées à AUTH_REQUIRED_PAGES, ce qui les rend accessibles à tout utilisateur connecté quel que soit son rôle. L'affichage est ensuite adapté au rôle via un flag `isDesigner / isArtisan`.

Bug historique corrigé : ces pages étaient initialement dans les sets rôle-spécifiques. Quand un client cliquait « Voir le projet » dans Mes Projets, le guard voyait `requiredRole = 'artisan'` mais `currentUser.role = 'client'` → il redirigeait silencieusement vers home, donnant l'impression que rien n'était cliquable.

7. Stores Zustand — état global

Quatre stores Zustand gèrent l'état global de l'application. Le store principal `useDedcoStore` centralise la navigation, l'authentification, le panier, les favoris et les scènes sauvegardées. Les trois autres stores sont spécialisés par domaine métier : briefs artisan, briefs designer, notifications. Tous persistent dans `localStorage` via le middleware `persist` de Zustand, ce qui permet à l'utilisateur de retrouver son état après un rafraîchissement.

Store	Fichier	Lignes	Persisté	Contenu
<code>useDedcoStore</code>	<code>lib/store.ts</code>	287	Oui (cart, favorites, currentUser)	Route, history, cart, favorites, currentUser, navigate, goBack
<code>useBriefArtisanStore</code>	<code>lib/artisan-brief-store.ts</code>	427	Oui	Briefs artisan, proposals, transitions, actions
<code>useBriefDesignerStore</code>	<code>lib/designer-brief-store.ts</code>	327	Oui	Briefs designer, prestations, paiements
<code>useNotificationStore</code>	<code>lib/notification-store.ts</code>	209	Oui	Notifications par rôle, unreadCount, markAsRead

Persistance sélective

Le store principal utilise `partialize` pour ne persister que les données utilisateur durables (cart, favorites, savedScenes, currentUser) — pas l'état UI éphémère (cartOpen, searchOpen, route). Cette stratégie évite les bugs de reprise où l'utilisateur se retrouve sur une page inattendue après un rafraîchissement. Les stores métier persistent intégralement leurs données pour préserver l'état des briefs en cours de rédaction.

Navigation programmatique

La navigation s'effectue via `navigate(route)` qui empile la route courante dans `history` avant de la remplacer. `goBack()` dépile la dernière entrée et y revient, ou redirige vers `home` si l'historique est vide. Ce mécanisme est utilisé par les boutons « Retour » des pages de détail.

8. Shell & layouts dashboard

Le shell de l'application alterne entre deux modes : **navbar publique** (pour les visiteurs et clients) et **sidebar dashboard** (pour les prestataires et admin connectés). La décision est prise dans `page.tsx` via le flag `isWrappedInDashboard`, qui masque alors la Navbar, le BottomNav et le Footer publics.

DashboardSidebar — mémorisée et stable

Le composant `DashboardSidebar` est encapsulé dans `memo()` pour éviter tout re-render inutile. Il reçoit `currentPage` et `children` en props — pas de souscription au store. Les 4 layouts (Artisan, Designer, Admin, Maison) ne souscrivent plus non plus au store : ils reçoivent `currentPage` du routeur parent. Cette architecture garantit que la sidebar ne soit jamais démontée entre deux pages d'un même dashboard — seul l'état actif des boutons change.

Layout h-screen + overflow-hidden

Le conteneur racine du dashboard utilise `flex h-screen overflow-hidden` plutôt que `min-h-screen`. Cette différence est cruciale : en mode `h-screen`, la sidebar est fixe (sticky implicite) et seul le contenu principal défile via `overflow-y-auto`. Les pages plein écran comme la messagerie peuvent alors gérer leur propre scroll interne sans impacter la sidebar ni le header mobile. Le header mobile a une hauteur fixe de `h-16` (64px) pour permettre aux pages de calculer leur hauteur exacte.

allowExitToSite — verrou prestataire

Une prop `allowExitToSite` contrôle l'affichage du bouton « Retour au site » dans la sidebar et la cliquabilité du logo. Pour les layouts Artisan, Designer et Maison, cette prop est à `false` — pas de bouton, logo non cliquable. Seul l'Admin garde `allowExitToSite=true` pour basculer entre son dashboard et le site public.

sharedPages — pages wrappées selon rôle

Les pages messages et notifications sont partagées entre tous les rôles. Quand un prestataire y accède, elles sont wrappées dans son layout dashboard (sidebar conservée). Quand un client y accède, elles s'affichent en mode page publique (navbar visible). Cette logique est implémentée dans le routeur via `sharedPages + isPrestataireRole`.

9. Moteur métier brief-artisan

Le module `src/lib/brief-artisan/` est le cœur métier de Dedco pour les briefs de fabrication artisanale. Il implémente une machine d'états complète sur 8 fichiers et ~1500 lignes de code typé. Le module est conçu pour être testable, prévisible et extensible — chaque transition est validée par des fonctions de permission, et chaque changement d'état déclenche des notifications automatiques.

Fichier	Lignes	Rôle
types.ts	184	Types BriefArtisanStatus, BriefArtisan, ArtisanBriefProposal, transitions
statutes.ts	133	Config des 11 statuts (label, color, badge, isUrgent)
transitions.ts	242	23 transitions autorisées + helpers isValidTransition
permissions.ts	79	13 fonctions canPublish, canSubmit, canPayDeposit, canCancel...
engine.ts	196	applyTransition, orchestration état + notifications
helpers.ts	148	formatBriefDate, getProgress, getNextAction, sortBriefs
notifications.ts	133	Génération de notifications par transition
mock.ts	313	Données mockées pour démo (5 briefs, 8 proposals)
index.ts	88	Barrel export

11 statuts de brief artisan

Le cycle de vie d'un brief artisan traverse 11 statuts, du brouillon initial jusqu'à la livraison finale et la clôture. Chaque statut a une configuration visuelle (couleur, badge, libellé) et détermine les actions disponibles pour chaque acteur (client, artisan, admin).

Statut	Libellé	Étape
DRAFT	Brouillon	Client rédige
SUBMITTED	Soumis	En attente modération
PUBLISHED	Publié	Visible par artisans, expiration 7j
AWAITING_DEPOSIT	Acompte requis	Artisan sélectionné, paiement 30%
IN_PROGRESS	En fabrication	Acompte payé, artisan travaille

Statut	Libellé	Étape
DELIVERED	Livré	Artisan a livré, validation client
COMPLETED	Terminé	Client a validé, paiement libéré
CANCELLED	Annulé	Annulation client/admin
EXPIRED	Expiré	Délai 7j dépassé sans proposition
REJECTED	Rejeté	Modération refusée
CHANGE_REQUEST_PENDING	Modif. en attente	Artisan demande modif client

10. Workflows métier

Dedco orchestre trois workflows principaux : le brief artisan (fabrication sur-mesure), le brief designer (prestation de design d'intérieur) et la commande catalogue (produit existant). Chaque workflow a son propre cycle de vie, ses propres paiements et ses propres pages de suivi. Les workflows se rejoignent sur la page unifiée « Mes projets » qui regroupe tous les éléments en cours pour le client.

Brief artisan — cycle complet

Le client crée un brief décrivant la pièce à meubler, les matériaux souhaités, le budget et les délais. Le brief est modéré, puis publié aux artisans qualifiés qui soumettent des propositions (devis + délai + approche). Le client sélectionne une proposition, paie un acompte de 30% (escrow), l'artisan fabrique, livre, et le client valide pour libérer le solde. À chaque étape, des notifications sont envoyées aux acteurs concernés, et des actions prioritaires sont mises en avant dans le dashboard client via ACTION_PRIORITY_ORDER.

Brief designer — prestation de service

Le brief designer suit un flux similaire mais adapté au service intellectuel : le client décrit son besoin (pièce, style, budget), un designer répond avec une proposition de mission (scope, honoraires, livrables). Après acceptation et paiement 100% (escrow), le designer produit ses livrables (moodboard, plan 2D/3D, sélection de sourcing). Le client peut demander jusqu'à 2 révisions incluses, puis valide la livraison pour libérer le paiement au designer.

Paiement escrow & garantie 1.5%

Tous les paiements transitent par un système d'**escrow** : le client paie 100% à la commande (ou 30% acompte + 70% à la livraison pour les briefs artisan), et Dedco retient les fonds jusqu'à validation de livraison. Une commission de 10% est prélevée au prestataire. Pour les projets artisan, un fonds de garantie de 1.5% est ajouté au paiement pour couvrir les litiges éventuels. Le paiement s'effectue via Mobile Money (MTN, Moov), les deux opérateurs dominants au Bénin.

Page Mes Projets — vue unifiée client

La page mes-projets-page.tsx regroupe tous les éléments en cours pour le client : briefs artisan avec propositions, projets en fabrication, prestations designer, paiements en attente, projets terminés et réclamations. Chaque carte affiche une priorité d'action (PAYMENT_REQUIRED, CHANGE_REQUEST_PENDING, DELIVERY_CONFIRMATION_REQUIRED, etc.) avec deadline et conséquence. Le tri automatique place les actions urgentes en haut via ACTION_PRIORITY_ORDER.

11. Pages partagées — affichage role-aware

Quatre pages de détail sont partagées entre client et prestataire. Elles utilisent le flag `isDesigner / isArtisan` pour adapter l'affichage au rôle de l'utilisateur connecté. Le prestataire voit le projet comme un **gestionnaire** (gérer ses livrables, traiter les révisions, attendre validation), tandis que le client voit le projet comme un **spectateur actif** (valider la livraison, demander des révisions).

Page	Vue designer	Vue client
projet-designer-detail	Mes livrables + Ajouter	Livrables du designer + Valider
projet-designer-detail (révisions)	Révisions à traiter + Marquer livrée	Demander une révision
projet-designer-detail (messages)	Client en interlocuteur	Designer en interlocuteur
projet-artisan-detail	Suivi fabrication	Suivi commande
brief-artisan-detail	Voir le brief reçu	Voir mon brief + propositions
brief-designer-detail	Voir la demande reçue	Voir ma demande

Helper `getBackToProjets` — boutons retour role-aware

Un helper centralisé `src/lib/back-to-projets.ts` mappe le rôle vers la route et le label du bouton retour. Il est utilisé par 12 boutons répartis sur 6 fichiers (`projet-artisan-detail`, `projet-designer-detail`, `brief-artisan-detail`, `brief-designer-detail`, `projet-paiement-artisan`, `designer-workflow-pages`).

```
export function getBackToProjets(role): { route, label } {
  switch (role) {
    case 'artisan': return { route: {page:'artisan-projets'}, label:'Projets en cours' };
    case 'designer': return { route: {page:'designer-projets'}, label:'Projets en cours' };
    case 'maison': return { route: {page:'maison-dashboard'}, label:'Mon espace' };
    case 'admin': return { route: {page:'admin-dashboard'}, label:'Mon espace' };
    default: return { route: {page:'client-projets'}, label:'Mes projets' };
  }
}
```

Carte partenaire — inversée selon rôle

Sur la page projet-designer-detail, la carte partenaire affiche le **client** quand l'utilisateur est designer (« Votre client »), et le **designer** quand l'utilisateur est client. L'avatar, le nom et le libellé s'adaptent. Cette inversion est essentielle pour que chaque acteur voie l'autre partie comme son interlocuteur, et non comme lui-même.

12. Messagerie & notifications

La messagerie Dedco est intégrée au dashboard pour les prestataires (sidebar) et accessible aux clients via le menu utilisateur. Elle utilise un layout 2 colonnes (liste des conversations + vue chat) qui s'adapte au mobile (un seul panneau à la fois). La hauteur est calculée via `h-full` dans un parent `h-screen + overflow-hidden`, ce qui permet au chat de gérer son propre scroll sans impacter la sidebar.

Bannière de sécurité — « Échange protégé par Dedco »

Une bannière `forest-pale` est affichée en bas de chaque conversation. Le texte a été renforcé pour être explicatif : « Réglez uniquement via le bouton Payer de Dedco. Ne partagez jamais de numéro Mobile Money, RIB ou lien de paiement externe dans cette conversation. En cas de doute, signalez le message à notre équipe. » L'icône `ShieldCheck` (positive) remplace l'ancienne `ShieldAlert` (anxiogène).

NotificationStore — notifications par rôle

Le store `useNotificationStore` maintient une liste de notifications typées (`brief_artisan`, `brief_designer`, `project`, `message`, `payment`, `delivery`, `review`, `system`, `litige`). Chaque type a une configuration visuelle (icône, couleur, fond). À la connexion, `initForRole(role)` peuple le store avec des notifications mockées adaptées au rôle de l'utilisateur — un artisan voit des notifications de briefs reçus, un client voit des notifications de paiements et livraisons. Le badge `unreadCount` alimente la cloche `NotificationBell` dans la navbar et les sidebars.

Marquer comme lu + Tout lire

Les notifications peuvent être marquées individuellement comme lues (clic → `markAsRead(id)`), ou toutes en une fois (`markAllAsRead()`). La page notifications affiche les éléments groupés par date (Aujourd'hui, Hier, Cette semaine, Plus ancien) avec un filtre Tout / Non lus. Le clic sur une notification déclenche `markAsRead` puis navigue vers la page associée (brief, projet, commande...).

13. Arborescence des fichiers

Le projet compte 154 fichiers TypeScript/TSX organisés selon une architecture en couches : couche app (routes), couche components (UI), couche lib (logique). Voici la structure synthétique avec les répertoires clés.

```
src/
├── app/
│   ├── api/                # Route API hello-world (1 fichier)
│   ├── layout.tsx          # Layout racine (fonts, metadata)
│   ├── page.tsx            # Page racine SPA (shell + DedcoRouter)
│   ├── loading.tsx         # Suspense fallback
│   ├── error.tsx           # Error boundary
│   └── not-found.tsx       # 404
├── components/
│   ├── dedco/
│   │   └── layout.tsx      # Navbar, BottomNav, Footer, UserMenu, NotificationBel
│   └──
│       ├── dedco-router.tsx # Routing SPA + guards + role-aware shell
│       ├── home-page.tsx   # Landing publique
│       ├── marketplace-page.tsx
│       ├── product-page.tsx
│       ├── cards.tsx       # ProductCard, ArtisanCard, DesignerCard
│       ├── cart-search.tsx  # CartSidebar + SearchOverlay (Radix Dialog)
│       ├── welcome-popup.tsx # Modal bienvenue + cookie banner
│       └── pages/
│           ├── (40 pages publiques + auth + client)
│           ├── artisan/    # 8 pages dashboard artisan
│           ├── designer/   # 8 pages dashboard designer
│           ├── admin/      # 8 pages dashboard admin
│           ├── shared-sidebar.tsx # DashboardSidebar memoisée
│           └── maison-dashboard.tsx
│   ├── ui/                 # shadcn/ui (40 composants)
│   └── theme-provider.tsx   # Dark mode (désactivé)
├── lib/
│   ├── store.ts            # useDedcoStore (287 lignes, persist)
│   ├── artisan-brief-store.ts
│   ├── designer-brief-store.ts
│   ├── notification-store.ts
│   ├── brief-artisan/     # Moteur métier (8 fichiers, 1516 lignes)
│   ├── types/             # Types métier (mes-projets, brief-artisan, etc.)
│   ├── mock/              # Données mockées
│   ├── dedco-data.ts       # Catalogue produits/artisans/designers
│   ├── dedco-status.ts     # Config statuts briefs + projets
│   ├── back-to-projets.ts  # Helper role-aware boutons retour
│   └── utils.ts            # Helpers communs
└── hooks/
    ├── use-mobile.ts
    └── use-toast.ts
```

14. Audit & corrections récentes

Le projet a fait l'objet de plusieurs vagues d'audit et de corrections. Voici la synthèse des 21+ corrections majeures appliquées, organisées par sévérité (P0 bloquant, P1 parcours, P2 UX, P3 polish). L'ensemble de ces corrections a été poussé sur GitHub et déployé sur Vercel en continu.

Catégorie	Corrections clés
P0 — Bloquants (6)	Persistence Zustand, route guarding par rôle, workflow brief fermé, modales Radix, contraste WCAG --text-3, error/loading/not-found
P0 — Cohérence rôles (6)	Bug isArtisan InvoicePage, retour OrderTracking selon rôle, déconnexion drawer, avatar mobile, bouton Voir le projet designer, boutons Marquer expédié + Voir détails artisan
P1 — Parcours (5)	MaisonLayout, NotificationBell mobile, profile mobile déconnexion, settings logout, notifications actions mobile
P2 — UX (10+)	Import mort order-tracking, profile dynamique, mes-projets designerId dynamique, boutons morts corrigés, dashboard boutons cliquables, designer-dashboard dynamique, badges hardcoded retirés, settings dynamique, messages filtrés, profile tab routing
Refonte sidebar stable	DashboardSidebar memoisée, layouts sans subscribe au store, allowExitToSite, h-screen+overflow-hidden, sharedPages pour messages/notifications
Séparation rôles stricte	Prestataires enfermés dashboard, redirection silencieuse (plus GuardBlocked), isPrestataireLocked, isPublicPage, ROLE_HOME_PAGE
Pages partagées role-aware	projet-*-detail et brief-*-detail retirés des sets rôle-spécifiques, helper getBackToProjets, carte partenaire inversée
Refonte messagerie	Layout 2 colonnes dans dashboard, h-full, marquer comme lu, bannière sécurité explicative, état vide refait
Kanban designer	DesignerProjectsPage en kanban (En cours / En revue / Terminés), cartes compactes avec milestone
Audit mobile complet	33 corrections padding p-6 → p-4 sm:p-6, overflow protection, min-w-0 sur flex, break-words sur montants FCFA

15. Roadmap & chantiers ouverts

Plusieurs chantiers restent ouverts pour faire passer Dedco d'un prototype fonctionnel à une plateforme production-ready. Voici les priorisations actuelles, classées par impact et complexité.

P1-4 — Migration vraies routes App Router

L'architecture SPA mono-route actuelle simplifie le développement mais limite le SEO, le partage d'URL et le server-side rendering. La migration vers de vraies routes App Router (/marketplace, /product/[id], /artisan/[id], /dashboard/artisan, etc.) permettrait le deep-linking, le prerendering des pages publiques, et la séparation propre entre layouts public et dashboard. C'est un gros chantier qui impacte 60+ pages et tous les appels navigate().

P1-6 — Consolidation modules data

Les données mockées sont actuellement réparties sur plusieurs fichiers (dedco-data.ts, dedco-data-expanded.ts, mock/mes-projets-data.ts, artisan-brief-mocks.ts, etc.) avec des IDs qui se chevauchent. Une consolidation en un seul module typé faciliterait la maintenance et la future migration vers une API réelle.

P3 — next/image sur 36 raw

36 balises natives subsistent dans le code au lieu de next/image. La migration permettrait l'optimisation automatique (WebP, lazy loading, responsive srcset) et réduirait le poids des pages de 30 à 50%.

Backend réel — Prisma + API routes

L'application fonctionne actuellement en mode démo avec données mockées et persistance localStorage. La mise en place d'un backend réel (Prisma + PostgreSQL + API routes Next.js) est nécessaire pour la production : authentification, persistance multi-appareils, gestion réelle des commandes et paiements Mobile Money. Le schema Prisma actuel (User, Post) est minimal et devra être étendu pour couvrir tout le modèle métier (Artisan, Designer, Product, Brief, Project, Order, Payment, Notification, Message, Review, Litige).

Tests & CI

Aucun test automatisé n'est en place. L'ajout de tests unitaires sur le moteur brief-artisan (transitions, permissions) et de tests E2E sur les parcours critiques (commande, brief, paiement) garantirait la non-régression lors des évolutions.

16. Conclusion

Dedco est aujourd'hui une application SPA fonctionnelle et robuste, qui démontre la faisabilité technique d'une marketplace multi-acteurs spécialisée sur l'aménagement intérieur béninois. L'architecture en couches (shell, router, stores, layouts, pages) offre une séparation claire des responsabilités et facilite l'évolution. Le moteur métier brief-artisan, avec sa machine d'états complète et ses 13 fonctions de permission, constitue le cœur intellectuel du produit et peut être réutilisé comme socle pour les workflows designer et maison déco.

La séparation stricte entre prestataires et clients, appliquée à la fois au niveau du shell et du route guarding, garantit une expérience claire : chaque acteur voit l'application à travers le prisme de son métier, sans confusion possible. Les pages partagées role-aware (projet-detail, brief-detail) démontrent qu'une même page peut s'adapter au contexte de l'utilisateur sans duplication de code.

Les prochaines itérations devront se concentrer sur la mise en production : migration vers de vraies routes App Router pour le SEO, backend réel avec Prisma et PostgreSQL, intégration Mobile Money, et tests automatisés. Le socle actuel — design system cohérent, stores typés, moteur métier validé — fournit une base solide pour ces évolutions. L'engagement qualité (audit mobile, refonte sidebar stable, séparation rôles) montre que la dette technique est gérée activement plutôt qu'accumulée.